

Multiplierless FIR Filter Design with Parallel-Cut Ellipsoid Method

Wai-Shing Luk

Abstract—A general-purpose multiplier is the most expensive arithmetic element in a digital finite impulse response (FIR) filter, so a filter whose coefficients are represented in canonical signed digit (CSD) form can be realized with a small number of adders and shifts instead of multipliers. The design problem, however, is awkward: the magnitude constraints are not convex in the impulse response, the coefficient quantizer is not convex at all, and the constraint set is effectively infinite because the frequency response must be bounded at every frequency. This article shows that all three difficulties are handled naturally by the ellipsoid method. Spectral factorization turns the magnitude specifications into convex constraints on the autocorrelation, the two-sided magnitude bounds are returned by the oracle as parallel cuts that shrink the search ellipsoid faster than a single cut, and the CSD quantizer is embedded in the oracle so that the optimizer reasons about a realizable coefficient vector rather than a relaxed one. The complete pipeline, from a frequency-domain specification to a synthesizable shift-and-add description, is described and implemented in three languages. Experiments report the iteration economy of the parallel-cut formulation and a cross-language comparison of the running time.

I. INTRODUCTION

A finite impulse response (FIR) filter computes a weighted sum of the recent input samples,

$$y[t] = \sum_{k=0}^{n-1} h[k] u[t-k],$$

and is specified by its frequency response $H(\omega) = \sum_{m=0}^{n-1} h(m)e^{-jm\omega}$. Its design consists of choosing the coefficients $h[0], \dots, h[n-1]$ so that $|H(\omega)|$ stays inside a prescribed mask. The classical tools for this task — windowing and frequency sampling [1], the Parks–McClellan exchange algorithm [2], METEOR [3], and peak-constrained least-squares design [4,5] — all assume that a full multiplier is available for each coefficient.

In application-specific integrated circuits and field-programmable gate arrays the multiplier is precisely the element one wants to avoid: it dominates area and power, and it becomes disproportionately expensive as the word length grows. A *multiplierless* filter replaces each product by a signed sum of shifted copies of the input. Because a shift by a power of two is free in dedicated hardware, a coefficient that is a sum of a few signed powers of two can be realized with a few adders and no multiplier [6–8].

The design problem is therefore not merely to find good coefficients, but to find coefficients that are *cheap to realize*.

Two obstacles stand in the way. First, the magnitude constraint

$$L(\omega) \leq |H(\omega)| \leq U(\omega), \quad \forall \omega \in [0, \pi], \quad (1)$$

is not convex in the coefficient vector $\mathbf{h}$; and it must hold at every frequency, so it is a semi-infinite constraint. Second, the set of coefficients representable with a bounded number of signed powers of two is discrete, and the natural procedure of optimizing first and rounding afterwards can destroy feasibility. This article addresses both by treating the whole design as a feasibility problem for the ellipsoid method, whose only requirement is a separation oracle.

The article is organized as follows. § II reformulates the magnitude specifications as convex constraints on the autocorrelation via spectral factorization. § III recalls the ellipsoid method and develops the parallel-cut update in the form used throughout. § IV introduces the canonical signed digit representation and the associated adder cost. § V embeds the quantizer in the oracle. § VI compares two spectral-factorization algorithms. § VII describes the implementation and reports cross-language results, and § VIII concludes.

II. DERIVING A CONVEX DESIGN PROBLEM

Work with the squared magnitude rather than the magnitude. The *autocorrelation* of the impulse response is

$$r[k] = \sum_i h[i] h[i+k], \quad k = 0, 1, \dots, n-1, \quad (2)$$

and the squared magnitude is its cosine series

$$R(\omega) = |H(\omega)|^2 = r[0] + 2 \sum_{k=1}^{n-1} r[k] \cos(k\omega) = \mathbf{a}(\omega)^\top \mathbf{r}, \quad (3)$$

where $\mathbf{a}(\omega) = (1, 2 \cos \omega, \dots, 2 \cos((n-1)\omega))^\top$. The map $\mathbf{r} \mapsto R(\omega)$ is affine, so the *squared* magnitude bounds

$$L^2(\omega) \leq R(\omega) \leq U^2(\omega), \quad R(\omega) \geq 0, \quad \forall \omega \in [0, \pi], \quad (4)$$

define a convex set in $\mathbf{r}$. This is the classical spectral-decomposition device of [9], building on the spectral factorization of [10]; the same convexification is surveyed in [11].

The problem solved in this article is

$$\begin{aligned} & \text{minimize} \quad \gamma, \\ & \text{subject to} \quad L^2(\omega) \leq R(\omega) \leq U^2(\omega), \quad \omega \in \Omega_p, \\ & \quad R(\omega) \leq \gamma, \quad \omega \in \Omega_s, \\ & \quad R(\omega) \geq 0, \quad \omega \in [0, \pi], \end{aligned} \quad (5)$$

where Ω_p and Ω_s are the passband and stopband, and γ bounds the stopband energy. The optimization variable is the autocorrelation $\mathbf{r}$; the frequency response is recovered afterwards by spectral factorization (§ VI). Because the constraints must hold on a continuum, and because the number of design variables is moderate while the number of constraints is large, this is exactly the regime in which the ellipsoid method is competitive with interior-point methods [12,13].

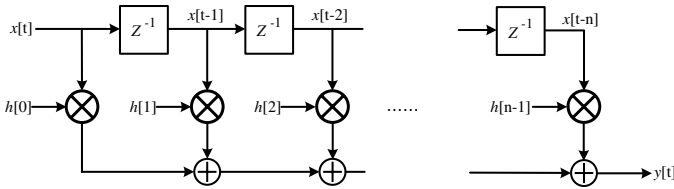


Figure 1: A typical structure of an FIR filter.

III. THE PARALLEL-CUT ELLIPSOID METHOD

A. Separation Oracles and Cutting Planes

Let $\mathcal{K} \subset \mathbb{R}^n$ be compact and convex. A *separation oracle* Ω , queried at a point x_0 , either certifies that $x_0 \in \mathcal{K}$ or returns a hyperplane that separates x_0 from $\mathcal{K}$:

$$g^\top(x - x_0) + \beta \leq 0, \quad \beta \geq 0, \quad g \neq 0, \quad \forall x \in \mathcal{K}. \quad (6)$$

The pair (g, β) is a *cutting plane*: it discards the half-space where $g^\top(x - x_0) + \beta > 0$. If $\beta = 0$ the cut is *central*, if $\beta > 0$ it is *deep*, and if $\beta < 0$ it is a *shadow cut*. When $\mathcal{K}$ is defined by $f_j(x) \leq 0$, a cut is obtained for free from a subgradient, $(g, \beta) = (\partial f_j(x_0), f_j(x_0))$; for a differentiable f_j the subgradient is the gradient. The method was introduced by Shor and by Yudin and Nemirovskii and used by Khachiyan to prove that linear programming is polynomial-time solvable [14,15].

B. The Ellipsoid Update

The search space is the ellipsoid

$$\mathcal{E} = \{x \mid (x - x_c)^\top Q^{-1}(x - x_c) \leq \kappa\}, \quad (7)$$

with center x_c , shape matrix $Q \succ 0$, and scale $\kappa > 0$. Writing

$$\tilde{g} = Qg, \quad \omega = g^\top \tilde{g}, \quad \tau = \sqrt{\kappa \omega},$$

every cut type applies the same update

$$x_c^+ = x_c - \frac{\rho}{\omega} \tilde{g}, \quad Q^+ = Q - \frac{\sigma}{\omega} \tilde{g} \tilde{g}^\top, \quad \kappa^+ = \delta \kappa, \quad (8)$$

and differs only in the scalar triple (ρ, σ, δ) . For a central cut ($\beta = 0$),

$$\rho = \frac{\tau}{n+1}, \quad \sigma = \frac{2}{n+1}, \quad \delta = \frac{n^2}{n^2-1}, \quad (9)$$

and for a deep cut with offset $\beta > 0$,

$$\begin{aligned} \rho &= \frac{\tau + n\beta}{n+1}, \\ \sigma &= \frac{2(\tau + n\beta)}{(n+1)(\tau + \beta)}, \\ \delta &= \frac{n^2}{n^2-1} \cdot \frac{\tau^2 - \beta^2}{\tau^2}. \end{aligned} \quad (10)$$

The application of the update to Q alone, rather than to a general positive definite matrix, saves n^2 floating-point operations per iteration; the split into κ and Q is what makes the scalar τ cheap to compute [15].

C. Parallel Cuts

The magnitude specifications (Eq. 1) are two-sided, and this is where the method pays off. Whenever a constraint has the form

$$l \leq a^\top x + b \leq u,$$

the oracle can return a *pair* of parallel cuts that share a normal g :

$$\begin{aligned} g^\top(x - x_c) + \beta_0 &\leq 0, \\ g^\top(x - x_c) + \beta_1 &\geq 0, \quad \forall x \in \mathcal{K}. \end{aligned} \quad (11)$$

The pair removes a slab from the ellipsoid rather than a half-space, which shrinks the volume more than either cut alone and therefore reduces the number of iterations [16]. A single deep cut is the special case in which one of the two planes is tangent.

The complete update uses the auxiliary quantities

$$\begin{aligned} \zeta_0 &= \tau^2 - \beta_0^2, \\ \zeta_1 &= \tau^2 - \beta_1^2, \\ \xi &= \sqrt{\zeta_0 \zeta_1 + \left(\frac{n}{2}(\beta_1^2 - \beta_0^2)\right)^2}, \end{aligned} \quad (12)$$

and, with $\eta = \tau^2 + n\beta_0\beta_1$,

$$\sigma = \frac{2\eta}{\tau^2 + \beta_0\beta_1 + \frac{n}{2}(\beta_0 + \beta_1)^2 + \xi}, \quad \rho = \sigma \cdot \frac{\beta_0 + \beta_1}{2}, \quad (13)$$

$$\delta = \frac{n^2}{(n^2-1)\tau^2} \left(\frac{\zeta_0 + \zeta_1}{2} + \frac{\xi}{n} \right). \quad (14)$$

This formulation is finite even when the two offsets are symmetric, $\beta_0 + \beta_1 = 0$, which is precisely the case that defeats a mean-offset expression dividing by $(\beta_0 + \beta_1)^2$. It also degenerates correctly: setting $\beta_1 = \tau$ makes the second plane tangent and reduces the parameters to the deep-cut values Eq. 10 with $\beta = \beta_0$. The intersection is empty when $\beta_0 > \beta_1$; no smaller ellipsoid exists when $\beta_0\beta_1 < -\tau^2/n$; and the update reduces to a deep cut when $\beta_1^2 > \tau^2$.

D. Volume Reduction

Applying the matrix-determinant lemma to Eq. 8,

$$\begin{aligned} \det Q^+ &= (1 - \sigma) \det Q, \\ \frac{\text{vol}(\mathcal{E}^+)}{\text{vol}(\mathcal{E})} &= \delta^{n/2} (1 - \sigma)^{1/2} \leq e^{-1/(2n)}. \end{aligned} \quad (15)$$

Algorithm 1 Cutting-plane optimization with parallel cuts

Require: oracle Ω , ellipsoid $\mathcal{E} \supseteq \mathcal{K}$, tolerance ϵ

Ensure: value γ^* , coefficients $\mathbf{h}_{\text{csd}}$

```

1:  $\gamma \leftarrow +\infty$ 
2: repeat
3:    $x_c \leftarrow \text{center}(\mathcal{E})$ 
4:    $(g, \beta_0, \beta_1, t) \leftarrow \Omega(x_c, \gamma)$ 
5:   if  $t < \gamma$  then
6:      $\gamma \leftarrow t$ ;  $\mathcal{E} \leftarrow \text{CENTRALCUT}(\mathcal{E}, g, 0)$ 
7:   else if  $\beta_1^2 \leq \tau^2$  then
8:      $\mathcal{E} \leftarrow \text{PARALLELCUT}(\mathcal{E}, g, \beta_0, \beta_1)$ 
9:   else
10:     $\mathcal{E} \leftarrow \text{DEEPCUT}(\mathcal{E}, g, \beta_0)$ 
11:   end if
12: until  $\text{vol}(\mathcal{E}) < \epsilon$ 
13: return  $\gamma^*, \mathbf{h}_{\text{csd}}$ 

```

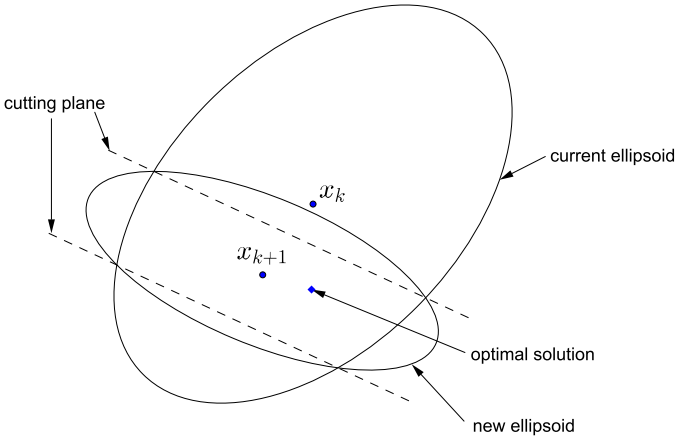


Figure 2: Parallel cuts.

After k iterations the volume is at most $e^{-k/(2n)}$ of the initial volume, so reaching a fraction ϵ requires $k \approx 2n \ln(1/\epsilon)$ iterations. The determinant of Q therefore decreases monotonically, and the factor δ stays in $(0, n^2/(n^2 - 1))$.

A robust implementation stores a factorization $Q = \kappa LDL^T$ with L unit lower triangular and D diagonal, and applies the rank-one update to (L, D) directly. Positive definiteness is then preserved by construction, which prevents the silent loss of definiteness that repeated rank-one downdates can cause in the unfactored form.

IV. CANONICAL SIGNED DIGIT AND ADDER COST

A coefficient is realized without a multiplier when it is a signed sum of powers of two,

$$c = \sum_j s_j 2^{e_j}, \quad s_j \in \{-1, 0, +1\}, \quad (16)$$

because each term is a shift and each pair of terms is an add or subtract. A canonical signed digit (CSD) representation is one in which no two adjacent digits are non-zero; it is unique and contains the fewest non-zero digits of any

Algorithm 2 CSD quantization with a non-zero-digit budget

Require: value x , budget nnz

Ensure: c with at most nnz non-zero digits

```

1:  $c \leftarrow 0$ ;  $b \leftarrow 2^{\lceil \log_2(1.5|x|) \rceil - 1}$ 
2: while  $\text{nnz} > 0$  and  $|x| > 0$  do
3:   if  $|1.5x| > b$  then
4:      $c \leftarrow c + \text{sign}(x)b$ 
5:      $x \leftarrow x - \text{sign}(x)b$ ;  $\text{nnz} \leftarrow \text{nnz} - 1$ 
6:   end if
7:    $b \leftarrow b/2$ 
8: end while
9: return  $c$ 

```

signed-digit representation of the same number [6]. For example,

$$28.5 = +00-00.+0_{\text{csd}} = 32 - 4 + 0.5,$$

which uses three non-zero digits rather than the four set bits of the binary form. A power of two, such as $h_i = 2^{-3}$, needs no adder at all.

The number of adders per coefficient is one less than the number of non-zero digits, before any sharing. Because several coefficients are realized simultaneously, the relevant problem is *multiple constant multiplication* (MCM): the goal is to build a network of adders and shifts that realizes all coefficients at minimum cost. *Common subexpression elimination* (CSE) searches for bit patterns shared across coefficients and reuses their computation, reducing the adder count and, importantly, the *adder depth* — the number of adder stages, which controls the critical path [7,8]. The number of non-zero digits is therefore a design parameter, not an accident of the coefficient values.

To expose the trade-off, a coefficient is quantized to the nearest CSD number with at most a prescribed number of non-zero digits, written

$$h_{\text{csd}}[k] = \text{csd}(h[k], \text{nnz}). \quad (17)$$

Enlarging the budget nnz improves the approximation and enlarges the set of feasible filters, at the price of more adders; shrinking it to one or two digits makes many otherwise realizable filters infeasible. The budget is a constraint of the design problem, and it is what makes the problem discrete.

A. Shift-Add Synthesis and Common Subexpressions

A CSD coefficient is realized by a shift for every non-zero digit and an add or subtract for every subsequent digit, so a coefficient with d non-zero digits costs $d - 1$ adders before any sharing. Two forms of sharing reduce this cost. Within a coefficient, a signed pattern that occurs at positions p and $q > p$ satisfies

$$\text{pat}_q(x) = \text{pat}_p(x) \gg (q - p),$$

so one shift-and-add network serves both occurrences, and the longest repeated substring of the digit string identifies

the pattern worth sharing. Across coefficients, the same idea selects the pattern that maximizes

$$\text{score} = (\text{nnz} - 1)(\text{occurrences} - 1),$$

which is exactly the number of adders saved. For a 64-tap design the two mechanisms typically remove between 40 and 60 percent of the adders; a 32-tap example required about 82 cells with sharing against roughly 110 without.

B. Filter Architecture

The same coefficient set admits a direct or a transposed realization. The direct form forms each product $h[k]x[n-k]$ and accumulates them; the transposed form pipelines the accumulator and applies the coefficients in reverse order, producing one output with a single-cycle latency. Both forms use the same CSD strings and therefore the same per-tap hardware, but they differ in register count, adder depth, and external wiring. The transposed form is the natural default for a self-contained module, whereas the direct form exposes the individual products for a custom accumulator tree or a polyphase decomposition.

V. QUANTIZATION-AWARE DESIGN

Designing a multiplierless filter is not a matter of optimizing first and rounding afterwards. At the relaxed optimum the active magnitude constraints are saturated, so the optimum lies on the boundary of the feasible set; the composite map $S^{-1} \circ \text{csd} \circ S$, which rounds a coefficient vector and returns its autocorrelation, is neither a projection nor exact, and even a small perturbation can push part of the passband below its lower bound. The rounded design can therefore be substantially infeasible.

The remedy is to move the quantizer inside the oracle. The pipeline is

$$\mathbf{r} \longrightarrow \mathbf{h} = S(\mathbf{r}) \longrightarrow \mathbf{h}_{\text{csd}} \longrightarrow \mathbf{r}_{\text{csd}} = S^{-1}(\mathbf{h}_{\text{csd}}), \quad (18)$$

where S is spectral factorization and S^{-1} recomputes the autocorrelation of the quantized response. The oracle is queried at a continuous center $\mathbf{r}$ and proceeds as follows.

- **Quantize.** Compute the continuous factor $\mathbf{h} = S(\mathbf{r})$ and replace every coefficient by its nearest CSD value under the budget: $h_{\text{csd}}[k] = \text{csd}(h[k], \text{nnz})$.
- **Re-evaluate exactly.** Recompute $\mathbf{r}_{\text{csd}} = S^{-1}(\mathbf{h}_{\text{csd}})$ and test the sampled constraints against $\mathbf{r}_{\text{csd}}$, not against $\mathbf{r}$. A feasible point then corresponds to a genuinely realizable coefficient vector.
- **Classify.** The candidate is feasible (improve the best-so-far value and take a central cut), violated (take a deep cut from the gradient at $\mathbf{r}_{\text{csd}}$), or ineffective (the cut does not shrink the ellipsoid).
- **Retry.** The ineffective case is peculiar to the discrete setting: because the admissible set is finite, several consecutive centers can map to the same CSD pattern, producing a cut that removes no new volume. The

Algorithm 3 Quantization-aware oracle Ω_Q

Require: center $\mathbf{r}$, budget nnz , retry flag

Ensure: a cut (g, β) , possibly an improved γ

```

1: if not retry and  $\mathbf{r} \notin \mathcal{K}$  then
2:   return cut at  $\mathbf{r}$ 
3: end if
4:  $\mathbf{h} \leftarrow \text{SPECTRALFACT}(\mathbf{r})$ 
5:  $\mathbf{h}_{\text{csd}} \leftarrow \text{CSDQUANTIZE}(\mathbf{h}, \text{nnz})$ 
6:  $\mathbf{r}_{\text{csd}} \leftarrow \text{INVERSESPECTRALFACT}(\mathbf{h}_{\text{csd}})$ 
7: if  $\mathbf{r}_{\text{csd}} \in \mathcal{K}$  then
8:    $\gamma \leftarrow f_0(\mathbf{r}_{\text{csd}})$ 
9:   return  $(\partial f_0(\mathbf{r}_{\text{csd}}), 0)$  ▷ central cut
10: else
11:   return  $(\partial f_j(\mathbf{r}_{\text{csd}}), f_j(\mathbf{r}_{\text{csd}}))$  ▷ deep cut, or retry
12: end if
```

oracle then perturbs the quantization decision, or re-derives the pattern from the updated center, for a bounded number of attempts.

One pitfall deserves emphasis. The oracle certifies feasibility only on the finite grid of $m = c_{\text{disc}}n$ sampled frequencies. Between samples the squared magnitude can dip below its lower bound even though the design passes at every sample; this is a *discretization artifact*, the familiar consequence of replacing a semi-infinite constraint by finitely many samples. It is mitigated by enlarging the sampling factor c_{disc} when the constraints are tight, and by re-checking the final design on a finer grid. The fine-grid check is advisory, because the optimizer never certified that resolution.

VI. SPECTRAL FACTORIZATION

Both the convex formulation and the oracle require a spectral factorization: given the autocorrelation Eq. 2, find the minimum-phase impulse response h that produces it. The minimum-phase factor is unique when r corresponds to a positive spectrum, and it is the factor a causal implementation stores.

Transform-based method (Kolmogorov). Evaluate the power spectrum $R(\omega)$ on an oversampled grid, form the log-magnitude $\alpha(\omega) = \frac{1}{2} \ln R(\omega)$, obtain the minimum-phase phase as its Hilbert transform $\phi = \mathcal{H}[\alpha]$, and return

$$h = \mathcal{F}^{-1}[\exp(\alpha(\omega) + j\phi(\omega))]. \quad (19)$$

The method is non-iterative, deterministic, and stable, and it has no tuning parameter; its costs are an FFT dependency and a memory footprint proportional to the oversampling factor, which is customarily 100 times the filter order.

Root-based method (Aberth–Ehrlich). The autocorrelation defines a palindromic polynomial whose roots occur in reciprocal pairs; all roots are found simultaneously by the Aberth iteration, the roots inside the unit circle are retained, and the factor is reconstructed from them. The method needs no FFT library, uses memory linear in the order, and exposes a convergence tolerance; convergence is

Algorithm 4 Multiplierless FIR design pipeline

Require: specification $(n, \omega_p, \omega_s, L, U)$, budget nnz
Ensure: a realizable coefficient vector $\mathbf{h}_{\text{csd}}$

```

1: initialize  $\mathcal{E}$  and  $\gamma \leftarrow +\infty$ 
2: repeat
3:    $\mathbf{r} \leftarrow \text{center}(\mathcal{E})$ 
4:    $(\mathbf{h}_{\text{csd}}, \text{cut}) \leftarrow \Omega_Q(\mathbf{r}, \text{nnz})$ 
5:    $\mathcal{E} \leftarrow \text{UPDATE}(\mathcal{E}, \text{cut})$ 
6: until  $\text{vol}(\mathcal{E}) < \epsilon$ 
7: return  $\mathbf{h}_{\text{csd}}$ 

```

not guaranteed for pathological inputs, and the smallest coefficients are recovered with somewhat lower relative accuracy.

Reconstruction from roots is sensitive to the order in which the linear factors are multiplied. Multiplying well-separated roots first — a Leja ordering — keeps the intermediate coefficients from growing, and reduces the reconstruction error from roughly 10^{-2} to machine precision. A second numerical point is the convergence test: an absolute residual $|P(z)|$ is not scale-free, and when the coefficients are large the threshold it implies can lie below the attainable floating-point floor, so the iteration never reports convergence. Scaling the polynomial by $s = \max(1, \max_k |a_k|)$ leaves the iteration invariant — both the correction and its denominator scale the same way — while making the tolerance relative, which restores convergence without changing the roots.

Which method is faster is a property of the implementation rather than of the algorithm. When the root finder is compiled and the FFT is not, the root-based method can be the faster choice; when both are compiled, the two methods are comparable and the crossover depends on the order. This observation motivates the cross-language experiments of § VII.

VII. IMPLEMENTATION AND EXPERIMENTS

A. The Software Pipeline

The pipeline of Eq. 18 has been implemented three times, in Python, Rust, and C++, from a single algorithmic specification. The heavy components are shared: an ellipsoid engine supplies the search space and the generic cutting-plane loop; a spectral-factorization routine supplies S ; and a CSD quantizer supplies $\text{csd}(\cdot, \text{nnz})$. The oracle checks the passband, stopband, and non-negativity constraints in round-robin order and returns the first violation it finds, which preserves the round-robin cursor that prevents the same cut from being reintroduced. The implementations are checked by property-based testing: a fast implementation is compared against a reference on randomly generated inputs, which exposes silent conversion and factorization defects that example-based unit tests miss.

B. Experimental Setup

The canonical design is an order- $n = 32$ lowpass filter with passband edge 0.12π , stopband edge 0.20π , a CSD budget of $\text{nnz} = 7$ non-zero digits, a discretization factor $c_{\text{disc}} = 15$, a tolerance of 10^{-14} , and an initial ellipsoid radius of 40. All three implementations read the same specification and share the same defaults; measurements were taken on a Windows x64 machine with release builds, five measured runs after two warm-up runs.

C. Cross-Language Results

Two observations stand out. First, the compiled implementations are within five percent of each other, and both are more than an order of magnitude faster than the interpreted one; the gap is the cost of the interpreter loop over the constraint checks, not the cost of the arithmetic. Second, the iteration counts differ across languages even though the inputs are identical. The ellipsoid method is a first-order method with no mechanism for converging to a unique optimum, so floating-point differences in the FFT libraries and in the accumulation order lead to different, equally valid trajectories; the three solutions satisfy the same specifications and have the same energy $\sum h[k]^2$.

D. Profiling

Profiling the interpreted implementation produced a counter-intuitive result: the spectral factorization accounted for only about six percent of the running time, while the constraint scan accounted for about eighty-three percent. The scan issued one short dot product per constraint — on the order of half a million tiny calls — and the interpreter dispatch, not the arithmetic, dominated. Replacing the per-constraint dot products with a single matrix-vector product followed by boolean masks, while preserving the round-robin cursor, made the scan primitive about 94 times faster and the end-to-end run about 1.6 times faster. After the change the run was dominated by library import time.

The same change did not transfer to C++. Porting the vectorization literally made the compiled implementation about 1.9 times *slower*, because the original compile-time per-row loop stops at the first violation and therefore performs about half the work of a full matrix-vector product. The lesson is that a vectorization win observed in an interpreted language must be re-measured after compilation; the measurement, not the conclusion, is what ports. Exposing raw pointers to the compiler and using a short chain of independent partial sums (rather than a single scalar reduction, which the compiler may not reassociate) recovered a further small margin in the compiled implementations.

E. Optimization and Sharing Results

The same kernel changes paid off very differently across the three languages:

Table I: Running time and iteration count for the order-32 design. The figures are indicative wall-clock times from the authors’ implementations.

Implementation	Mean time	Relative to C++	Iterations
C++ (FFTW3)	303 ms	1.00×	1850
Rust (reallfft)	286 ms	0.95×	2530
Python (NumPy)	4046 ms	13.4×	1693

Table II: Effect of the kernel changes in three languages.

Change	Language	Before	After	Speedup
Vectorized constraint scan	Python	3.05 s	0.082 s	37×
End-to-end after the scan	Python	3.01 s	1.87 s	1.6×
Raw-pointer dot product	C++	0.172 s	0.160 s	1.07×
Four-accumulator dot	Rust	23.3 ns	10.1 ns	2.3×
Oracle scan	Rust	13.9 μ s	10.7 μ s	1.3×

The CSD coefficient set is compressed further by common-subexpression sharing:

A 32-tap design then needs about 82 cells with sharing, against roughly 110 without, a reduction of about 25 percent.

F. Multiplierless Result

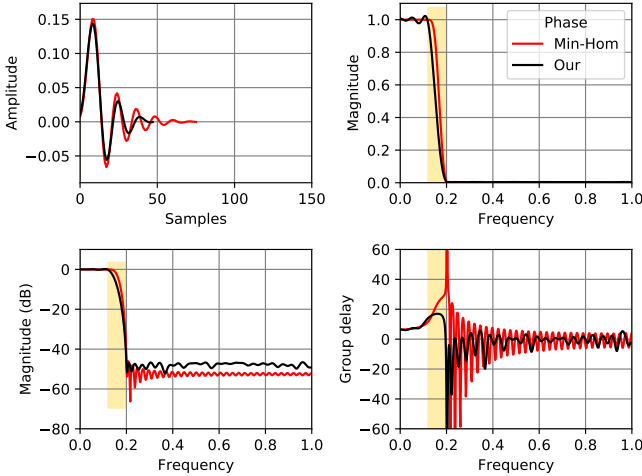


Figure 3: Magnitude response of a lowpass design.

The output of the pipeline is the quantized coefficient vector $\mathbf{h}_{\text{csd}}$ together with its adder network. Because the quantizer was consulted during the optimization rather than after it, the returned design satisfies the sampled specifications by construction, and the only remaining verification is to confirm that the design also holds between the samples of the grid. The shift-and-add description is then emitted directly for synthesis.

VIII. CONCLUDING REMARKS

The ellipsoid method is an unusually good fit for multiplierless FIR design. Its separation oracle accommodates the semi-infinite magnitude constraints without ever enumerating them; the two-sided nature of those constraints is returned as parallel cuts, which reduce the iteration count; and the discrete CSD quantizer can be embedded in the oracle so that the optimizer searches over realizable

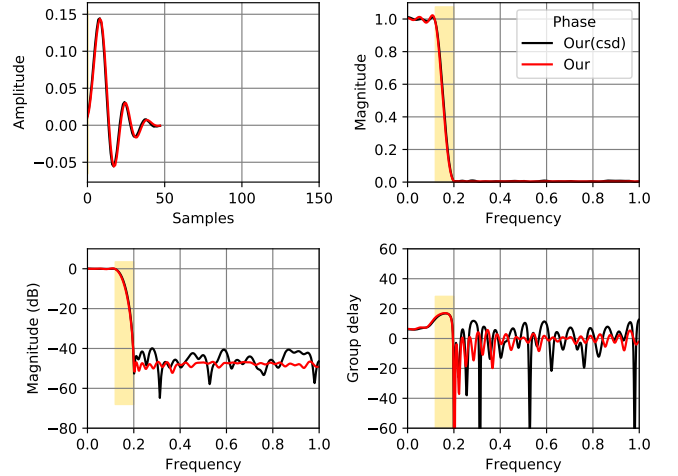


Figure 4: Magnitude response after CSD quantization.

coefficient vectors rather than over a relaxation. The same formulation applies unchanged to the robust and parametric variants of the design problem, and the numerical experiments confirm that a compiled implementation of the whole pipeline runs in a fraction of a second for a representative order-32 design.

Three directions remain open. The first is a sharper analysis of the parallel cut under a discretization grid, so that the fine-grid advisory check can be replaced by a guarantee. The second is to fold the adder-sharing stage — the multiple-constant-multiplication network that realizes the CSD coefficients — into the oracle, so that adder cost, and not only coefficient error, is a first-class design objective. The third is a systematic study of the filter architecture: the direct and transposed forms realize the same transfer function but differ substantially in adder depth and register count, and the choice interacts with the coefficient ordering that the CSD quantizer produces.

IX. ACKNOWLEDGMENTS

The author is grateful to the contributors of the open-source packages on which this work depends.

Table III: Adder count with and without common-subexpression sharing.

CSD pattern	Flat adders	Shared adders	Saving
+0-0+0-0	4	2	50%
+0-0+0-0+0-0	6	2	67%
+00-00+00-00	4	2	50%

REFERENCES

- [1] A.V. Oppenheim, R.W. Schaffer, J.R. Buck, others, Discrete-time signal processing, Prentice hall Englewood Cliffs, NJ, 1989.
- [2] T.W. Parks, J.H. McClellan, Chebyshev approximation for nonrecursive digital filters with linear phase, Circuit Theory, IEEE Transactions On CT-19 (1972) 189–194.
- [3] K. Steiglitz, T.W. Parks, J.F. Kaiser, METEOR: A constraint-based FIR filter design program, Signal Processing, IEEE Transactions On 40 (1992) 1901–1909.
- [4] I.W. Selesnick, M. Lang, C.S. Burrus, Constrained least square design for FIR filters without specified transition bands, Signal Processing, IEEE Transactions On 44 (1996) 1879–1892.
- [5] J.W. Adams, J.L. Sullivan, Peak-constrained least-squares optimization, Signal Processing, IEEE Transactions On 46 (1998) 306–321.
- [6] G.W. Reitwiesner, Binary Arithmetic, Advances in Computers 1 (1960) 231–308.
- [7] H. Samueli, An improved search algorithm for the design of multiplierless FIR filters with powers-of-two coefficients, Circuits and Systems, IEEE Transactions On 36 (1989) 1044–1047.
- [8] D. Marković, R.W. Brodersen, DSP architecture design essentials, Springer, 2012.
- [9] S.-P. Wu, S. Boyd, L. Vandenberghe, FIR filter design via spectral factorization and convex optimization, in: Applied and Computational Control, Signals, and Circuits, Springer, 1999: pp. 215–245.
- [10] T.N.T. Goodman, C.A. Micchelli, G. Rodriguez, and S. Seatzu, Spectral factorization of laurent polynomials, Advances Comput. Math. 7 (1997) 429–454.
- [11] T.N. Davidson, Enriching the art of FIR filter design via convex optimization, Signal Processing Magazine, IEEE 27 (2010) 89–101.
- [12] S. Boyd, L. Vandenberghe, Convex optimization, Cambridge university press, 2009.
- [13] S. Boyd, C. Barratt, Ellipsoid method, Lecture Slides on Class EE364b of Stanford University, Available: [Http://www.stanford.edu/Class/Ee364b/Lectures/Html](http://www.stanford.edu/Class/Ee364b/Lectures/Html) (2008).
- [14] R.G. Bland, D. Goldfarb, M.J. Todd, The ellipsoid method: A survey, Operations Research 29 (1981) 1039–1091.
- [15] R.G. Bland, D. Goldfarb, M.J. Todd, The ellipsoid method: A survey, Operations Research 29 (1981) 1039–1091.
- [16] J.B.G. Frenk, J. Gromicho, S. Zhang, A deep cut ellipsoid algorithm for convex programming: Theory and applications, Mathematical Programming 63 (1994) 83–108.